

**COMPUTER SCIENCE DEPARTMENT
UNIVERSITY OF CRETE**



Undergraduate Thesis

**Optimizing Indoor Positioning Accuracy through Sensor Fusion
Techniques**

Michail Samaritakis

Email: csd4529@csd.uoc.gr

Supervisor: **Kostas Magoutis**

Additional Supervisor / Committee Member: **Chrysostomos Zeginis**

June 2026

Table of Contents

1. Abstract	2
2. Introduction	2
3. Technological Background	3
3.1 Indoor Positioning and Analytics Technologies	3
3.2 Location Estimation Algorithms	3
3.3 Data Visualization	4
4. Related Work and Technical Context	4
5. System Design and Implementation	7
5.1 System Overview and Architecture	7
5.2 Calibration and Coordinate System	8
5.3 BLE Localization Pipeline	9
5.4 Sensor Fusion and PDR Pipeline	9
5.5 Dashboard and Data Visualization	10
5.6 Logging, Export, and Report Generation	12
6. Evaluation, Results, and Discussion	12
6.1 Evaluation Methodology	12
6.2 Static Ground-Truth Setup	13
6.3 Aggregate Accuracy Results	13
6.4 Per-Point Result Analysis	15
6.5 Walking Route Discussion	17
6.6 Practical Transmitter Setup and Evaluation Limitations	17
7. Conclusions	17
8. Future Work	18
9. References	18
Appendix A. Evaluation Metrics and Supporting Tables	20
Appendix B. Configuration Summary	20
Appendix C. Full current_test.json	22
Appendix D. Source Code References and Selected Snippets	24
Appendix E. Repository and Submission Notes	28

1. Abstract

This thesis presents OIPASFT (Optimizing Indoors Positioning Accuracy using Sensor Fusion Techniques), an Android indoor-positioning prototype designed to investigate whether sensor fusion can improve indoor location behavior in a small calibrated indoor environment. The system scans Bluetooth Low Energy (BLE) beacon signals, converts RSSI measurements into distance estimates, computes a BLE-based position using weighted trilateration/multilateration, and combines that estimate with smartphone motion-sensor data through pedestrian dead reckoning and a 2D Kalman-style fusion filter.

The implementation is a native Android application with no remote backend. It includes BLE scanning through the Kontakt.io Android SDK, smartphone sensor handling, a live room dashboard, local CSV/JSON experiment logging, and PowerShell scripts for pulling device logs and generating thesis-ready CSV reports. The current implementation supports BLE-only, PDR-only, and fused tracking modes, and it distinguishes tracker reset anchors from evaluation-eligible ground-truth samples.

The final quantitative evaluation uses static ground-truth samples only. Walking routes are retained for qualitative discussion because they do not include continuous independent ground-truth. Across 46 evaluation-eligible static samples from 9 valid static sessions, the fused/rendered position achieved a mean error of 0.7245 m and an RMSE of 0.8543 m. The raw accepted BLE baseline for the same evaluation samples achieved an RMSE of 0.8911 m, so the fusion pipeline produced an approximate 4.1% RMSE reduction. These claims are reported cautiously because the dataset is small and unevenly distributed across static points.

2. Introduction

Indoor positioning is the problem of estimating the location of a user or device inside a building or enclosed space. Unlike outdoor positioning, where satellite-based systems are often available, indoor environments create practical challenges such as signal attenuation, multipath propagation, limited line of sight, and changing radio conditions. As a result, indoor-positioning systems often rely on local infrastructure, mobile device sensors, or a combination of both.

The purpose of this thesis is to design and implement an Android prototype that investigates whether sensor fusion can improve the stability and usability of indoor positioning. The prototype combines BLE RSSI-based localization with phone-based motion sensing. BLE provides absolute position corrections when enough valid transmitter signals are available, while PDR provides relative movement estimates between BLE fixes. A Kalman filter then combines these two sources into a final rendered position.

The implemented use case is a general indoor positioning and lab experiment prototype. It is not a production navigation system and it is not connected to a cloud backend. Instead, it is designed as a controlled thesis testbed where calibration parameters, beacon placement, RSSI behavior, PDR behavior, and fusion outputs can be recorded and evaluated.

The topic was selected because indoor positioning remains difficult in GPS-denied environments and because low-cost smartphone-based systems are attractive for practical experimentation. The practical goal of the work was to build a working Android testbed that can show, log, and evaluate how BLE-based absolute positioning and smartphone motion sensing behave when combined in a single local indoor-positioning prototype.

The main contributions of the work are:

- A native Android prototype that performs BLE scanning, RSSI ranging, BLE multilateration, PDR, and Kalman fusion locally on the device.
- A calibration-driven room model using JSON files for room dimensions, beacon identities, beacon coordinates, RSSI calibration values, and algorithm parameters.
- A live dashboard for inspecting fused position, raw BLE position, tracking mode, beacon counts, missing beacons, anchors, and diagnostic values.
- Local CSV/JSON logging designed to preserve experiment context and support thesis evaluation.
- A script-based workflow for pulling Android logs and generating summary/sample CSV files and SVG traces for analysis.

3. Technological Background

3.1 Indoor Positioning and Analytics Technologies

The system uses BLE RSSI ranging as the absolute positioning technology. BLE transmitters periodically advertise packets that can be received by a smartphone. The received RSSI value is used as a noisy indication of signal strength, and the system converts this value into an estimated distance. This approach is low-cost and works with common mobile devices, but RSSI is sensitive to obstacles, body position, antenna orientation, transmitter power, and multipath effects. [4], [5]

The prototype does not use WiFi positioning. It also does not use a remote backend or external positioning server. All scanning, filtering, localization, fusion, visualization, and logging are handled by the Android application itself.

The phone sensors used by the implementation include the Android step detector, step counter, rotation vector or game rotation vector, gyroscope, linear acceleration, gravity, accelerometer, magnetometer, and pressure sensor when available. These sensors are used to estimate heading, motion state, detected steps, and relative movement between absolute BLE fixes. [13], [14]

3.2 Location Estimation Algorithms

The BLE ranging model uses the log-distance path-loss equation:

$$d = 10^{((RSSI_{1m} - RSSI) / (10 * n))}$$

In this equation, d is the estimated distance in meters, $RSSI_{1m}$ is the calibrated RSSI value measured at one-meter, $RSSI$ is the current measured signal strength, and n is the path-loss exponent. In the current configuration, $n = 2.0$ is used as a manually chosen baseline value rather than an experimentally optimized value. The value remains configurable in the calibration JSON so that it can be replaced later if controlled calibration data supports a better value. [5]

For BLE position estimation, the implementation uses weighted least-squares multilateration. At least three valid BLE transmitters are required for a two-dimensional solve, and more than three can be used when available. The solver selects the closest valid transmitter as the reference, constructs linearized circle subtraction equations, and weights each equation according to the estimated distance. A minimum effective distance of 0.85 m is used in the weighting function to prevent a near-field RSSI estimate from dominating the solution. [5], [6]

PDR estimates relative movement using detected steps, a configured step length, and a corrected heading. For each detected step, the system computes room-coordinate displacement as $dx = stepLength * \sin(correctedHeading)$ and $dy = stepLength * \cos(correctedHeading)$. The $correctedHeading$ is calculated as $rawAndroidHeading + headingOffsetDegrees$. The raw Android

heading is obtained mainly from the game rotation vector or rotation vector sensors, with fallback support from accelerometer and magnetometer data when needed. [7], [8]

The heading offset is required because the Android orientation estimate and the custom room map do not naturally share the same coordinate frame. Android reports device/world orientation according to the phone sensor convention, while the thesis map uses xMeters across grid columns, yMeters across grid rows, and an origin at row 0, col 0. Without this rotation correction, a physical step in one room direction could be rendered as movement in a rotated or incorrect map direction.

In the active calibration, headingOffsetDegrees is set to 146.0 degrees. This value is reported as an empirically tuned calibration parameter for the tested phone-holding style and room-coordinate convention. It was not automatically learned by the app and it is not treated as a universal device-independent constant. The parameter affects PDR and the fused Kalman trajectory because it controls relative movement direction, but it does not affect BLE-only trilateration.

The final fused position is produced by a two-dimensional constant-velocity Kalman filter. Manual anchors reset the filter to a known position, PDR steps predict relative motion, and accepted BLE fixes correct the predicted position. The filter maintains x and y axis position and velocity states independently. [8], [9], [11]

3.3 Data Visualization

The application includes a live dashboard that displays status information and a room map. The room is represented as a 5x5 grid mapped to meter coordinates. The dashboard shows the current rendered position, the BLE-only coordinate when available, the current reference/anchor coordinate, beacon positions, live and filtered beacon counts, missing beacons, selected tracking mode, trial label, recording state, and adaptive tuning state.

The dashboard is intended for experiment inspection and runtime diagnosis rather than as the sole source of final statistical reporting. Final reporting is based on exported CSV files and generated SVG/PNG evidence outputs, which are used in the Results section and appendices.

4. Related Work and Technical Context

This chapter summarizes the technical literature and documentation that are most relevant to OIPASFT. The project is a smartphone-based indoor-positioning prototype that combines BLE RSSI ranging, weighted multilateration, pedestrian dead reckoning, and Kalman-filter-based sensor fusion. Therefore, the selected references focus on smartphone indoor positioning, BLE signal behavior, RSSI to distance conversion, trilateration/multilateration, PDR, Kalman filtering, and the Android/Kontakt.io implementation environment.

4.1 Smartphone Based Indoor Positioning

Indoor positioning is commonly studied because satellite-based positioning is unreliable inside buildings, where signals are blocked, attenuated, or heavily affected by walls and indoor materials. Smartphone-based systems are attractive because the phone already contains radio interfaces and motion-sensors that can be used without specialized user equipment. Davidson and Piche provide a broad smartphone oriented survey of indoor positioning methods, including WLAN/BLE approaches, inertial sensors, map-aided techniques, and hybrid methods [1]. Zafari et al. provide a wider survey of indoor localization systems and compare technologies such as WiFi, RFID, UWB, Bluetooth, and

RSS based methods [2]. Subedi et al. focus specifically on smartphone indoor positioning using RF-based technologies, including WiFi and BLE [3].

These surveys support the design choice of using a smartphone as the central device in OIPASFT. They also justify the decision to avoid relying on a single positioning source: BLE provides absolute but noisy position evidence, while phone sensors provide continuous relative motion but accumulate drift.

4.2 BLE RSSI Ranging and Log Distance Path Loss

BLE beacons are widely used in indoor localization because they are low-cost, battery-powered, and easy to deploy. However, RSSI values are strongly affected by multipath propagation, conditions with limited line of sight, body blocking, antenna orientation, beacon battery state, and nearby objects. Faragher and Harle show that BLE signals are affected by fast-fading and that practical BLE localization requires careful filtering and measurement handling [4].

The project converts RSSI values into distance estimates using the log-distance path-loss model. This model relates the RSSI measured at a known reference distance, usually 1 m, to the current RSSI and a path-loss exponent that represents the propagation behavior of the environment. Pakanon et al. use BLE RSS measurements, a channel/path-loss model, and trilateration to estimate a mobile phone position, and their results show that averaging multiple RSS measurements can reduce distance error [5]. This directly supports the use of RSSI averaging in OIPASFT before distance estimation.

4.3 Trilateration, Multilateration, and Post Processing

Trilateration estimates a receiver position from distances to known transmitter positions. In ideal 2D trilateration, three circles intersect at a single point. In real indoor BLE positioning, the RSSI-derived distances contain errors, so the circles usually do not meet cleanly. For this reason, least-squares or weighted least-squares methods are commonly used to estimate the most likely position from noisy distances. Pakanon et al. apply least-squares trilateration because practical BLE distance estimates do not produce a perfect intersection [5].

OIPASFT implements weighted least-squares multilateration rather than relying only on exact trilateration. The solver uses accepted beacon distances, chooses a reference beacon, and weights equations so that distance estimates from closer beacons contribute more while avoiding excessive dominance from unstable near-field RSSI. This is consistent with the literature's view that raw trilateration normally requires filtering, correction, or post-processing. De Blas and López de Ipiña also argue that simple BLE trilateration benefits from post-processing and corrective logic, including filtering of unstable estimates and preventing positions from being rendered outside physically valid areas [6].

4.4 Pedestrian Dead Reckoning with Smartphone Sensors

Pedestrian dead reckoning estimates a user's new position from a previous position, a heading direction, and an estimated displacement or step length. It is useful because it can continue producing relative motion estimates even when absolute radio fixes are temporarily unavailable. However, PDR suffers from accumulated error: small heading or step length errors grow over time. Harle's survey explains the main categories of pedestrian inertial positioning systems and concludes that dead reckoning benefits from periodic absolute corrections from another positioning source [7].

In OIPASFT, PDR is initialized or corrected by manual anchors and accepted BLE fixes. The Android implementation uses sensors such as the step detector, step counter, rotation vector or game rotation vector, gyroscope, linear acceleration, gravity, accelerometer, magnetometer, and pressure when available. Android documentation confirms that the Android platform provides motion-sensors such as gravity, linear acceleration, rotation vector, step counter, and step detector, while accelerometer and gyroscope sensors are hardware based [13]. The Android sensor overview also documents rotation vector and magnetic field sensor types that are relevant to orientation estimation [14].

4.5 Kalman Filter Based Sensor Fusion

Sensor fusion combines multiple imperfect measurements in order to produce a more stable position estimate. In this thesis, BLE measurements are absolute but noisy, while PDR is smooth and continuous but drifts. A Kalman filter is appropriate for this situation because it separates prediction from correction: the prediction step propagates the current state using a motion model, while the correction step incorporates a measurement according to its uncertainty. Kalman's original paper introduced the mathematical basis of linear filtering and prediction [11].

Several indoor positioning studies support this fusion strategy. Liu et al. implemented a real-time Android indoor positioning workflow that combines iBeacon/BLE positioning, PDR, and Kalman filtering, reporting that fusion improved or stabilized localization performance compared with BLE-only or PDR-only operation in their tests [8]. R besaat et al. propose BLE indoor localization using Kalman based fusion of trilateration and dead reckoning, which is close in spirit to the OIPASFT architecture [9]. Jamil et al. also discuss IMU based indoor navigation and Kalman-filter-based sensor fusion, although their main focus is learning based prediction rather than BLE/PDR fusion [10].

4.6 Android and Kontakt.io Implementation Context

The implementation runs as a native Android application. Android's Bluetooth documentation explains that BLE scanning requires appropriate Bluetooth and, depending on Android version and use case, location related permissions [12]. Android BLE scanning documentation also describes the startScan callback based scanning workflow used by Android BLE applications. OIPASFT additionally uses the Kontakt.io Android SDK: Kontakt.io documentation describes ProximityManager as the entry point for ranging and monitoring BLE devices, and the configuration documentation explains that ProximityManager should be configured before scanning [15], [16].

4.7 Relation to OIPASFT

Compared with the cited work, OIPASFT is a small-scale Android prototype rather than a large deployment. It does not use WiFi positioning or fingerprinting. Instead, it focuses on a defensible experimental pipeline: BLE RSSI readings are smoothed and converted into distances, accepted distances are solved through weighted multilateration, unstable BLE fixes are rejected by gates and residual checks, PDR provides continuous motion between absolute fixes, and a constant-velocity 2D Kalman filter fuses BLE corrections with PDR movement. This makes the project most closely related to the BLE/PDR/Kalman fusion studies, while the surveys provide the broader technical context.

Final experimental results are presented in Chapter 6, while this chapter remains focused on related work and technical context.

Table 1 summarizes how the most relevant cited work relates to OIPASFT. The table separates broad survey papers, BLE ranging/trilateration work, PDR work, BLE/PDR fusion studies, and implementation documentation. This makes clear that the thesis is closest to the BLE, PDR, and Kalman-filter fusion studies, while remaining a smaller Android prototype rather than a large deployment.

Table 1. Related-work comparison summary.

Source	Main focus	Relevant contribution	Relation to OIPASFT
[1]-[3]	Smartphone and indoor-localization surveys	Compare indoor-positioning technologies such as WLAN/WiFi, BLE, RSS, RFID, UWB, inertial sensing, and hybrid methods.	Provide the broader motivation for using a smartphone-based hybrid approach. OIPASFT is narrower because it implements only BLE RSSI, PDR, and fusion.
[4]	BLE beacon signal behavior and fingerprinting	Shows that BLE RSSI is affected by indoor propagation effects and needs careful measurement handling.	Supports the need for RSSI smoothing and conservative BLE acceptance. OIPASFT does not implement fingerprinting; it uses RSSI-to-distance ranging.
[5]	BLE RSSI ranging and trilateration	Uses RSS measurements, a path-loss model, averaging, and trilateration for indoor positioning.	Directly supports the log-distance model, RSSI averaging, and BLE multilateration part of OIPASFT.
[6]	Improved BLE trilateration	Discusses corrective logic for BLE trilateration, including handling unstable estimates and invalid positions.	Supports the use of residual rejection, boundary checks, and post-processing before accepting a BLE fix.
[7]	Pedestrian inertial positioning survey	Explains PDR principles and the drift problem in pedestrian inertial positioning systems.	Supports using PDR only as a relative-motion source that needs periodic anchors or BLE corrections.
[8]	Android iBeacon/PDR/Kalman workflow	Combines iBeacon/BLE positioning, smartphone sensors, PDR, and Kalman filtering in a real-time Android context.	Closest practical reference for the OIPASFT fusion pipeline, although OIPASFT is evaluated in a smaller controlled room.
[9]	BLE localization with Kalman-based fusion	Combines BLE trilateration and dead reckoning using Kalman-based fusion.	Closest algorithmic reference for fusing BLE corrections with dead-reckoning movement.
[10]	IMU-based indoor navigation and prediction	Discusses IMU-based indoor navigation and Kalman-filter-related sensor fusion ideas.	Relevant for sensor-fusion context, but less directly aligned because OIPASFT focuses on BLE/PDR fusion rather than learning-based prediction.
[12]-[16]	Android and Kontakt.io implementation documentation	Documents Android BLE permissions, Android sensors, and Kontakt.io ProximityManager usage.	Supports the implementation choices and runtime permission/scanning requirements of the native Android prototype.

5. System Design and Implementation

5.1 System Overview and Architecture

The system is mainly a native Android application referred to in this thesis as OIPASFT. It coordinates BLE scanning, calibration loading, BLE positioning, sensor fusion, dashboard rendering, experiment controls, and local log export. There is no remote backend. The app stores experiment evidence locally as timestamped CSV and JSON files, which are later copied from the Android device into the project TestLogs folder.

At runtime, OIPASFT loads the active calibration, scans configured BLE transmitters, estimates a BLE position when enough accepted signals are available, predicts relative movement with PDR, and produces the displayed position through Kalman-based fusion. The detailed step-by-step runtime sequence and file-level responsibility table are provided in Appendix D.

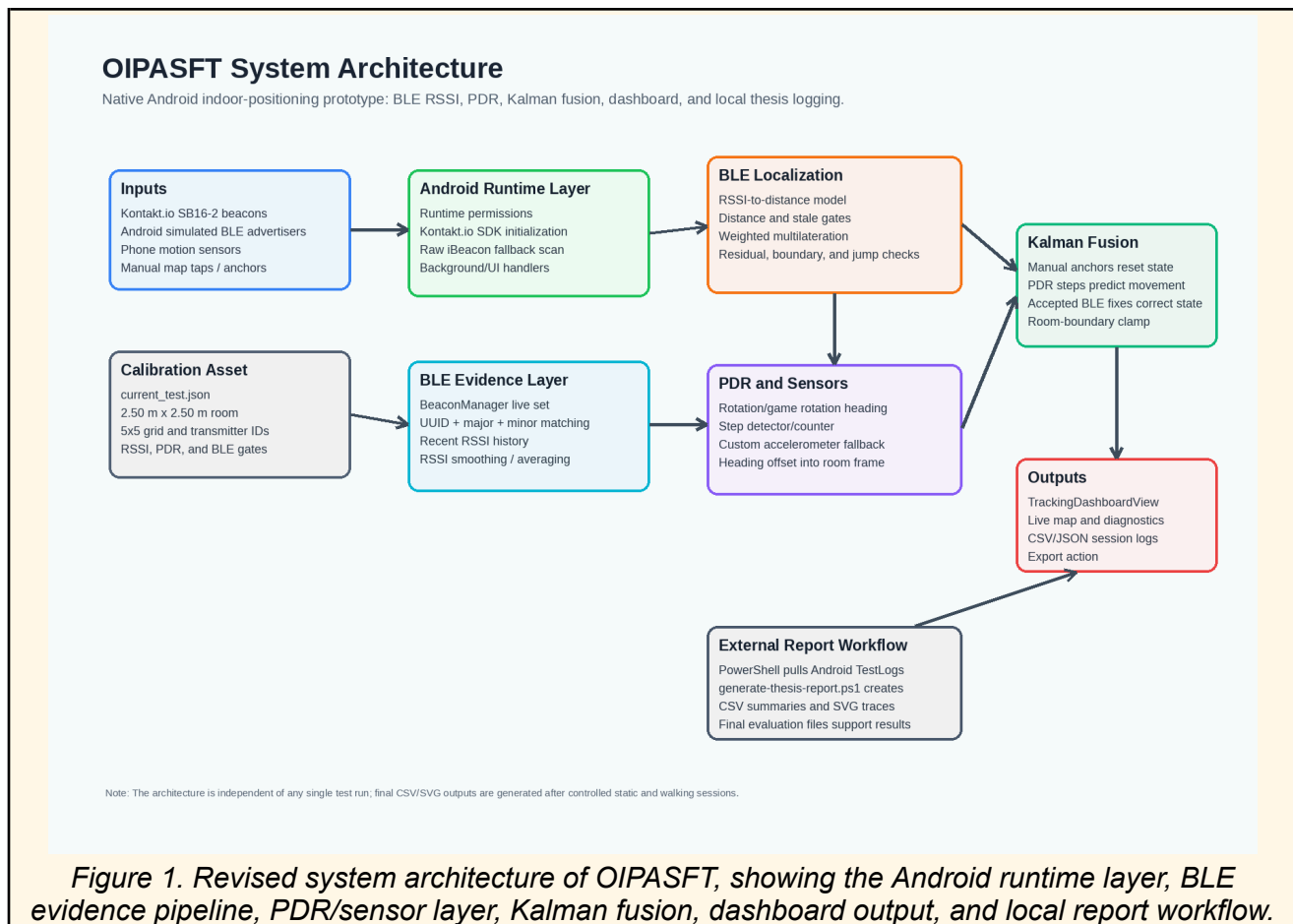


Figure 1. Revised system architecture of OIPASFT, showing the Android runtime layer, BLE evidence pipeline, PDR/sensor layer, Kalman fusion, dashboard output, and local report workflow.

5.2 Calibration and Coordinate System

The active calibration is stored in current_test.json. It defines the 2.50 m by 2.50 m room, the 5x5 map grid, the configured BLE transmitter identities, the room-coordinate convention, and the main BLE and PDR parameters. The full parameter list and beacon definitions are not repeated in the main body; they are provided in Appendices B and C.

In the app coordinate system, grid columns map to xMeters and grid rows map to yMeters, with row 0, col 0 corresponding to $x = 0$ and $y = 0$. The current implementation is two-dimensional and does not use beacon height in the positioning calculation.

The heading offset is kept as a calibration parameter because Android sensor heading and the custom room map do not share the same coordinate frame. In the final calibration, headingOffsetDegrees is set to 146.0 degrees. The calculation logic and justification for this value are included in Appendix B.

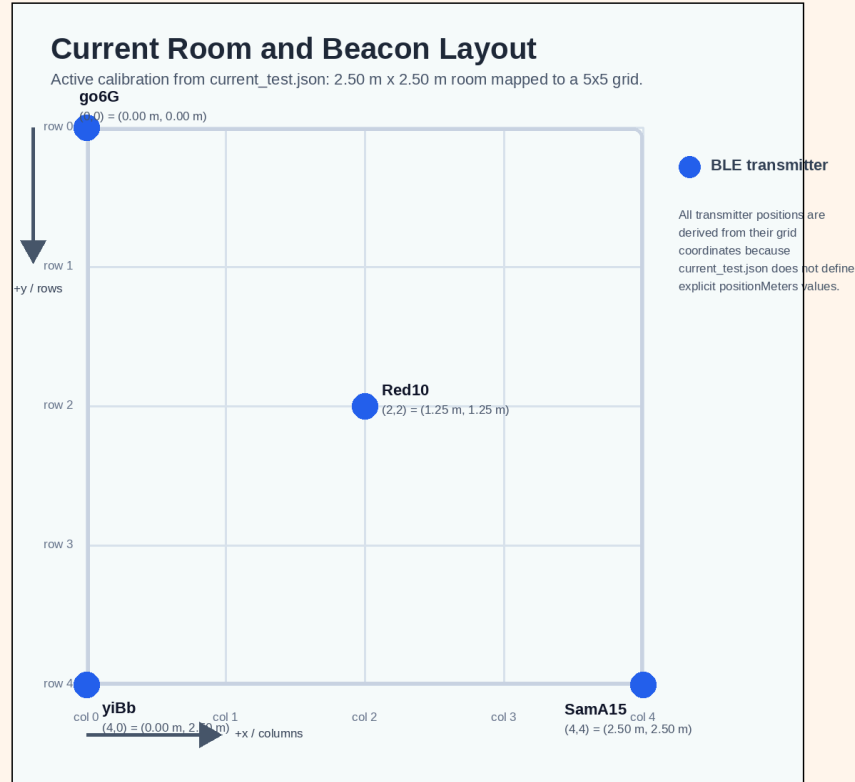


Figure 2. Active 5x5 BLE transmitter layout generated from current_test.json for the 2.50 m x 2.50 m calibration area.

5.3 BLE Localization Pipeline

BLE localization provides the absolute-position component of the prototype. The application accepts only configured transmitter identities, keeps recent RSSI evidence for each live transmitter, smooths the RSSI values, and converts them to distance estimates using the log-distance model described in Chapter 3. When at least three valid transmitters remain, weighted multilateration is used to estimate the BLE position.

Because RSSI-derived distances are noisy indoors, a BLE solve is not accepted automatically. The implementation applies distance gates, residual checks, room-boundary validation, and jump rejection before a BLE fix is passed to the fusion component. The detailed thresholds and gate

values are kept in Appendix D so that the main chapter focuses on the algorithmic flow rather than configuration constants.

5.4 Sensor Fusion and PDR Pipeline

The sensor-fusion pipeline combines absolute BLE corrections with relative movement from smartphone motion sensing. PDR estimates short-term movement from step events, step length, and the corrected heading, while the Kalman filter smooths the final rendered position by combining PDR predictions with accepted BLE fixes.

The heading offset is treated as a calibration parameter because the Android heading frame and the room map frame are not naturally aligned. Manual anchors initialize or reset the tracker state and are logged as trace context, but they are excluded from accuracy evaluation because they directly force the position to a selected coordinate. Detailed PDR parameters, heading-offset justification, and fusion input roles are provided in Appendices B and D.

5.5 Dashboard and Data Visualization

The dashboard is implemented as a lightweight Android View based interface in OIPASFT. The current Android build displays the app label as OIPASFT. It displays system status, selected tracking mode, reason/status strings, live and filtered beacon counts, missing beacons, sample count, trial label, recording state, adaptive tuning state, live position, BLE position, reference/anchor coordinate, and a room map.

The dashboard supports BLE-only, PDR-only, and Fusion-modes. BLE-only renders accepted BLE positions directly when available. PDR-only relies on the existing fused/PDR state and requires an initialized origin, which is why the UI can show PDR_WAITING_FOR_ANCHOR before a map tap or accepted fix initializes the tracker. Fusion-mode is the main operating mode and uses the Kalman filtered position as the rendered output. The dashboard also includes experiment controls for recording state, trial labels, adaptive tuning, and log export.

The live dashboard does not draw a continuous path trail in the current implementation. Path history is stored in logs and then drawn by the report-generation script as SVG path traces.

Android Runtime Permission Prompts in the OIPASFT Build

The current Android build requests permissions for BLE/location, physical activity sensors, and nearby-device scanning.

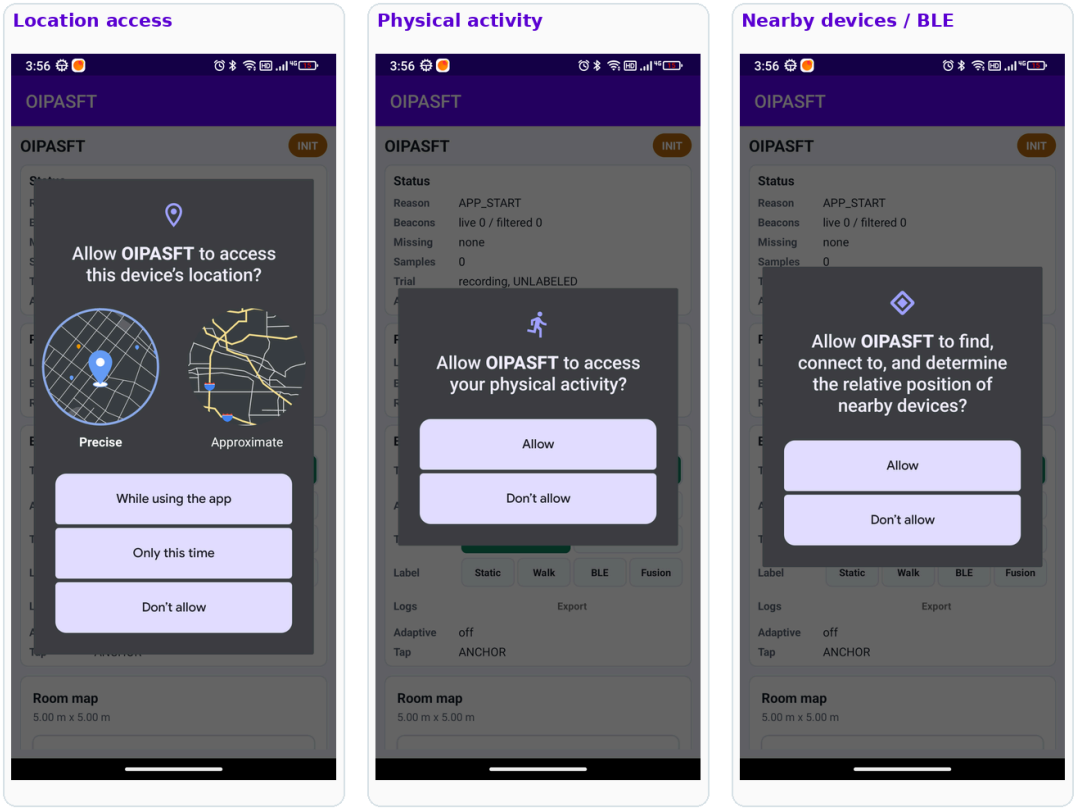


Figure 3. Updated Android runtime permission prompts from the OIPASFT build for precise location access, physical activity sensing, and nearby device/BLE operation.

OIPASFT Dashboard States Used During Thesis Testing

The dashboard exposes tracking mode, beacon status, recording controls, labels, adaptive tuning, and the room map.

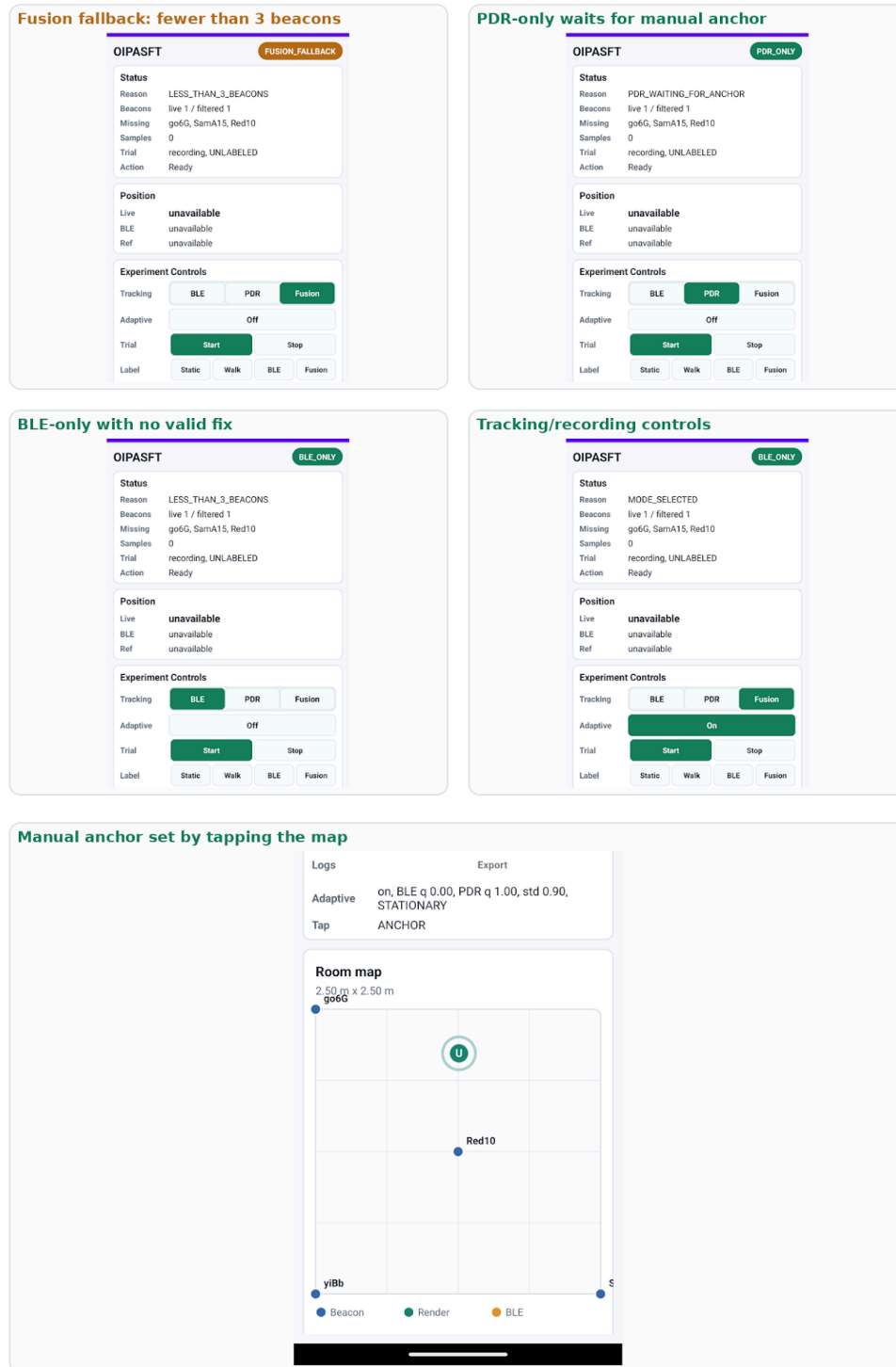


Figure 4. Updated OIPASFT dashboard states, including Fusion fallback, PDR-only waiting for a manual anchor, BLE-only operation without a valid fix, experiment controls, and manual anchor set by tapping the map.

Screenshot note: The captured screenshots document current UI behavior before final experiments. Some images show Android runtime permission prompts and dashboard fallback states. The manual anchor screenshot should be interpreted as a manual anchor set by tapping the map, not as an independent accuracy sample.

5.6 Logging, Export, and Report Generation

The system writes local thesis evidence through UserPositionHistory. Each app run produces timestamped CSV and JSON logs that preserve rendered positions, available BLE coordinates, ground-truth fields when available, diagnostic values, recording labels, and evaluation eligibility flags.

After the experiment runs, PowerShell scripts pull the Android TestLogs folder with adb and generate thesis-ready CSV summaries and SVG traces. The detailed log schema, generated file list, and script responsibilities are documented in Appendix D.

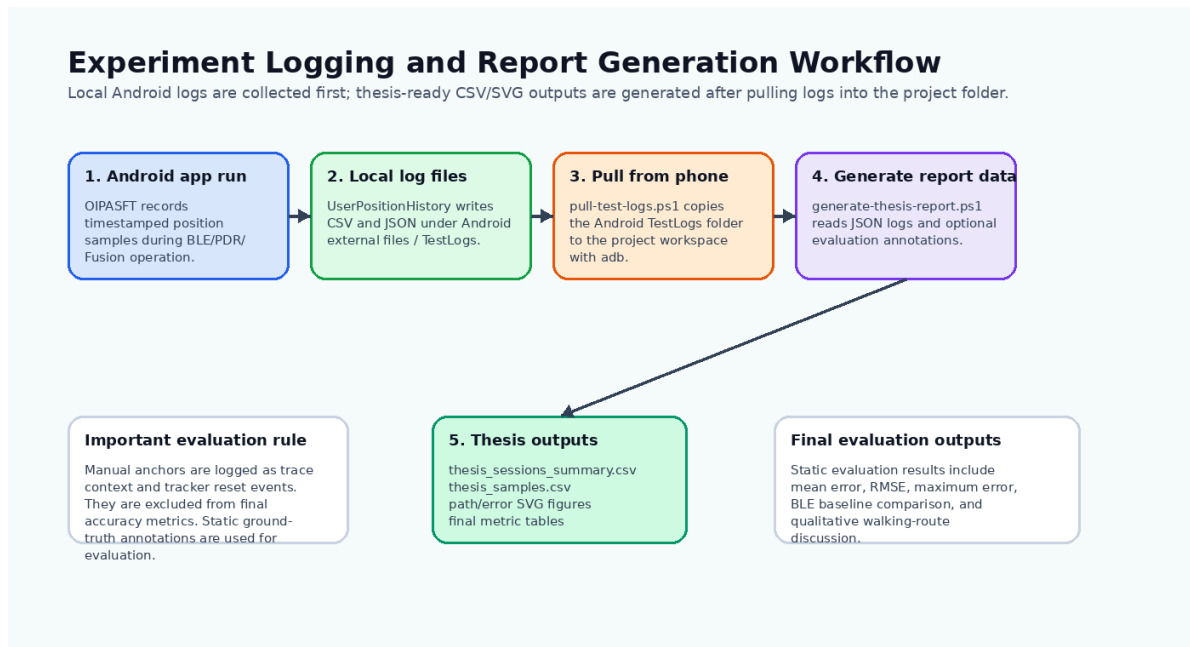


Figure 5. Logging, export, and report-generation workflow. Runtime CSV/JSON logs are pulled from the phone and converted into thesis-ready CSV summaries and SVG traces.

6. Evaluation, Results, and Discussion

6.1 Evaluation Methodology

The final controlled evidence set is complete. Quantitative evaluation is based only on static ground-truth samples, because those samples have known independent coordinates. Walking route sessions are retained only for qualitative visualization and discussion, since continuous ground-truth was not recorded along the routes. In total, 28 logged sessions and 241 logged samples were produced, but only 9 sessions contained valid static evaluation samples and were used for quantitative accuracy metrics.

6.2 Static Ground-Truth Setup

The final calibrated room was a 2.50 m x 2.50 m test area. Five static ground-truth points were marked inside the room and three repeated static attempts were recorded for each point, producing 15 static attempts. Manual anchors were used only for tracker initialization or reset and were explicitly excluded from accuracy metrics because a map tap directly forces the tracker state to the selected coordinate.

6.3 Aggregate Accuracy Results

The static evaluation produced 46 evaluation-eligible samples across 9 valid static sessions. The raw BLE baseline is the accepted raw BLE position recorded for the same evaluation-eligible Fusion-mode static samples. It is therefore a useful baseline for comparison, but it is not described as a fully separate repeated BLE-only experiment.

Table 2. Final static ground-truth points.

Point ID	x (m)	y (m)	Description
P1	0.50	0.50	Lower left / origin side interior point
P2	2.00	0.50	Lower right interior point
P3	1.25	1.25	Center point near Red10
P4	0.50	2.00	Upper left interior point
P5	2.00	2.00	Upper right interior point

Table 3. Final static trial validity summary.

Category	Sessions / attempts	Reason or role in the evaluation
Valid static sessions	9 sessions: P1_R3; P2_R1 and P2_R2; P3_R1; P4_R1 and P4_R2; P5_R1 to P5_R3	Included in final static accuracy results.
Rejected static sessions	6 sessions: P1_R1 and P1_R2; P2_R3; P3_R2 and P3_R3; P4_R3	Excluded because they produced no valid static evaluation sample, had insufficient accepted beacon support, or had no usable samples.
Manual anchor samples	truthSource = MANUAL_ANCHOR	Excluded from accuracy metrics because map taps reset the tracker state.

Table 4. Final aggregate static accuracy results.

Metric	Fused / rendered position	Raw accepted BLE baseline
Evaluation eligible samples	46	46
Mean error	0.7245 m	0.7554 m

RMSE	0.8543 m	0.8911 m
Maximum error	1.9315 m	1.9353 m
RMSE reduction relative to raw BLE	4.1%	Reference baseline

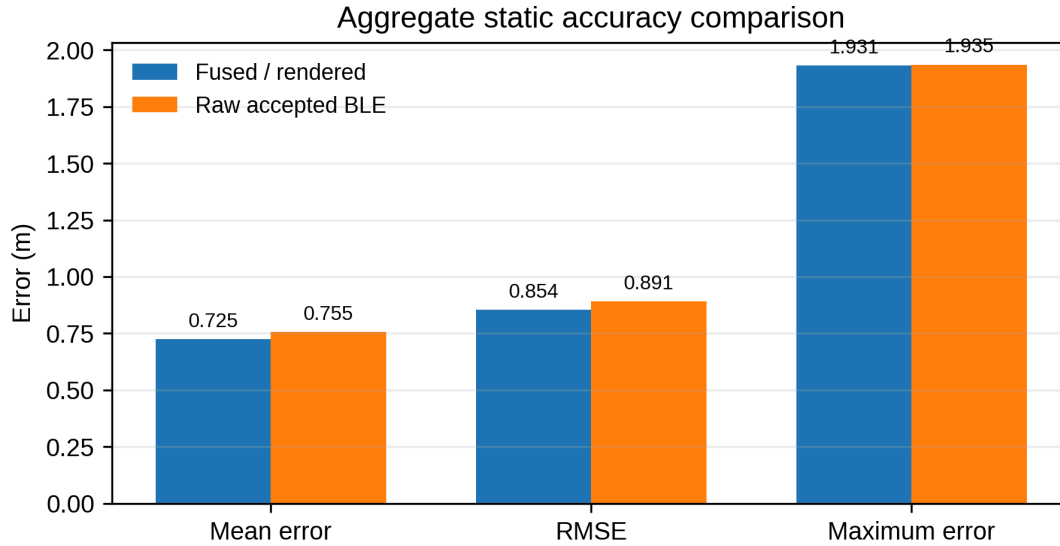


Figure 6. Aggregate static accuracy comparison between the fused/rendered output and the raw accepted BLE baseline.

Figure 6 shows a modest improvement from the fused/rendered position over the raw accepted BLE baseline. The fused RMSE is 0.8543 m compared with 0.8911 m for the raw accepted BLE values, corresponding to an approximate 4.1% RMSE reduction. This improvement is reported cautiously because the dataset is small, the accepted samples are unevenly distributed across points, and the BLE baseline is derived from accepted raw BLE estimates inside the Fusion-mode evaluation logs.

6.4 Per-Point Result Analysis

The static results were strongest at P5, where all three repeated runs produced valid evaluation samples and the fused RMSE was 0.4950 m. P1 had the highest error, with a fused RMSE of 1.5576 m, while P3 contained only one valid evaluation sample. This uneven distribution is important: the aggregate metrics summarize the final accepted evidence, but they are not interpreted as a balanced large-scale benchmark. Several attempted sessions at P1, P2, P3, and P4 were rejected because the system had fewer than three accepted transmitters, no accepted BLE fix, or no usable samples.

Table 5. Per-point static evaluation summary.

Point	Samples	Fused mean (m)	Fused RMSE (m)	Fused max (m)	Raw BLE RMSE (m)
P1	9	1.5331	1.5576	1.9315	1.6214
P2	3	0.8535	0.8596	0.9976	0.8729
P3	1	0.8638	0.8638	0.8638	0.8751
P4	3	0.6765	0.6893	0.8483	0.6739
P5	30	0.4692	0.4950	0.7240	0.5307

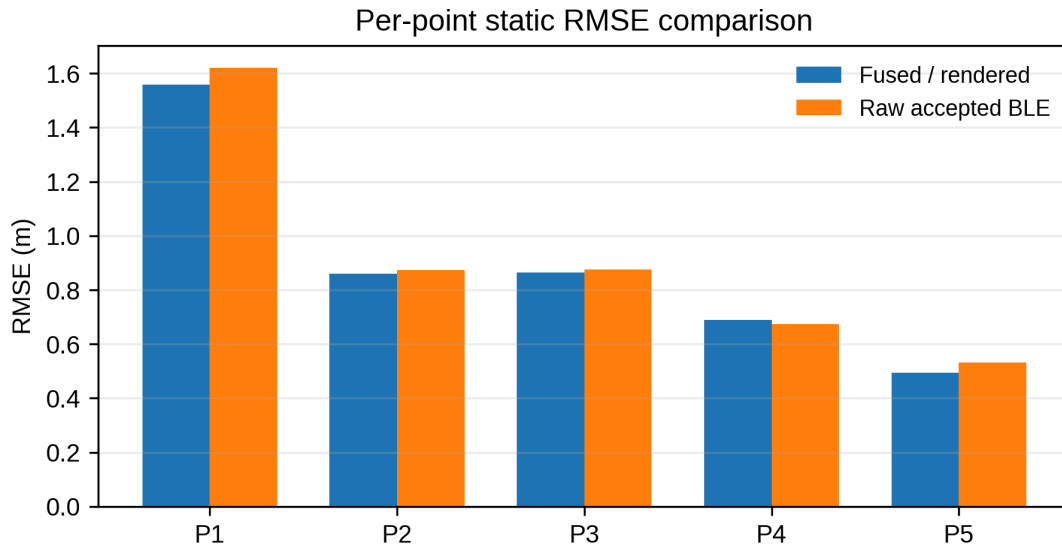


Figure 7. Per-point static RMSE comparison between the fused/rendered output and the raw accepted BLE baseline.

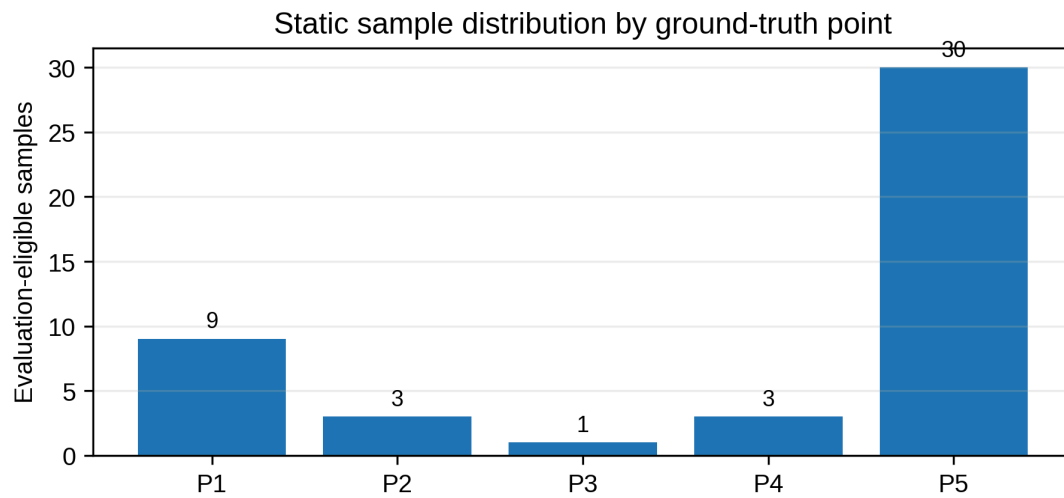


Figure 8. Distribution of evaluation-eligible static samples across the five ground-truth points.

Figures 7 and 8 highlight why the aggregate result should be interpreted cautiously. The fused output improves RMSE at most points, but the number of valid samples is uneven: P5 contributes 30 of the 46 evaluation-eligible samples, while P3 contributes only one sample.

6.5 Walking Route Discussion

Table 6. Walking route qualitative summary.

Route	Path	Attempts	Use in report
Route A	P1 (0.50, 0.50) to P3 (1.25, 1.25) to P5 (2.00, 2.00)	3 attempts; BLE fix ratios 0.2000, 0.3125, and 0.3333	Valid for qualitative discussion of path behavior and intermittent BLE support.
Route B	P4 (0.50, 2.00) to P3 (1.25, 1.25) to P2 (2.00, 0.50) to P5 (2.00, 2.00)	3 attempts; BLE fix ratios 0.0000, 0.0000, and 0.0909	Rejected or diagnostic only because accepted BLE support was absent or too sparse.

Because the walking routes do not have continuous independent ground-truth, they are not used for numerical RMSE or mean error claims. Route A is retained as qualitative walking evidence with limited BLE coverage. Route B is treated as rejected/diagnostic evidence because BLE support was absent or too sparse. The walking logs still help demonstrate how the dashboard, recording controls, fallback state, and path visualization behave when the system is used during motion.

Adaptive tuning exists in the prototype, but it was kept disabled for the final controlled evidence set. It is therefore reported as an implemented capability that requires a separate controlled experiment before any accuracy improvement claim can be made.

6.6 Practical Transmitter Setup and Evaluation Limitations

The final evaluation used the Ulefone Armor 22 as the scanner device and a transmitter set composed of configured BLE identities, physical Kontakt.io beacons where available, and Android phones acting as simulated BLE advertisers. The detailed device roles and Android versions are listed in Appendix E.

A major practical limitation was intermittent BLE availability. During testing, some transmitters were dropped or not discovered consistently by the scanner even at short ranges of roughly 1 m to 5 m. This behavior is reported conservatively because it may involve several factors, including BLE advertising intervals, Android scanning behavior, Kontakt.io SDK behavior, phone power management, transmitter configuration, RSSI volatility, and the filtering gates used by the prototype.

When fewer than three transmitters passed the filtering pipeline, the system could not produce a valid BLE multilateration fix and either fell back to the fused/PDR state or reported that no position fix was available. The development process also involved practical configuration challenges around Android Studio, Gradle, Kontakt.io SDK setup, and beacon account/cloud synchronization, which motivated the emphasis on a frozen calibration file and a reproducible local logging workflow.

7. Conclusions

This thesis presented OIPASFT, an Android based indoor-positioning prototype that integrates BLE RSSI measurements, smartphone motion sensing, pedestrian dead reckoning, and Kalman-style

fusion. The implementation provides live visualization, multiple tracking modes, manual initialization, structured CSV/JSON logging, and reproducible report-generation scripts.

The system successfully loads a calibrated room layout, scans known BLE beacon/transmitter identities, estimates positions, displays live tracking on a room map, supports BLE-only, PDR-only, and Fusion-modes, records structured CSV/JSON logs, and generates thesis-ready evidence outputs. It also separates manual anchors from evaluation samples, which prevents tracker reset events from being counted as independent accuracy evidence.

In the final static evaluation, the fused position achieved a mean error of 0.7245 m and an RMSE of 0.8543 m across 46 evaluation-eligible samples. The raw accepted BLE baseline on the same evaluation samples had an RMSE of 0.8911 m, so the fusion approach produced an approximate 4.1% RMSE reduction. This supports a cautious positive conclusion: the implemented fusion pipeline slightly improved the accepted static evidence while the broader evaluation still shows that BLE availability and calibration quality strongly affect practical indoor positioning performance.

8. Future Work

- Evaluate the system in larger and more complex indoor environments.
- Use a more stable physical beacon setup with more BLE transmitters to improve geometry, redundancy, and BLE fix availability.
- Collect more balanced static evaluation data with more repeated samples at every ground-truth point.
- Record continuous independent ground-truth for walking routes so that walking RMSE and path error can be evaluated quantitatively.
- Experimentally calibrate the path-loss exponent and RSSI at one-meter values for each transmitter and environment.
- Automate or formalize heading offset calibration using known straight-line walking segments or two-point room anchors.
- Improve PDR by calibrating step length per user/device and by further testing step detection under different phone carrying positions.
- Evaluate the adaptive tuning mode as a separate controlled experiment before claiming any accuracy benefit from it.
- Run independent BLE-only repeated trials to create a stronger baseline comparison against Fusion-mode.

9. References

The bibliography is formatted in IEEE numbered style and includes academic sources for indoor localization, BLE RSSI positioning, PDR, Kalman filtering, and implementation documentation for Android and Kontakt.io components.

- [1] P. Davidson and R. Piche, "A Survey of Selected Indoor Positioning Methods for Smartphones," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 1347 to 1370, 2017, doi: 10.1109/COMST.2016.2637663.
- [2] F. Zafari, A. Gkelias, and K. K. Leung, "A Survey of Indoor Localization Systems and Technologies," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2568 to 2599, 2019, doi: 10.1109/COMST.2019.2911558.
- [3] S. Subedi, G. R. Kwon, S. Shin, S. S. Hwang, and J. Y. Pyun, "A Survey of Smartphone Based Indoor Positioning System Using RF Based Wireless Technologies," *Sensors*, vol. 20, no. 24, Art. no. 7230, 2020, doi: 10.3390/s20247230.
- [4] R. Faragher and R. Harle, "Location Fingerprinting With Bluetooth Low Energy Beacons," *IEEE Journal on Selected Areas in Communications*, vol. 33, no. 11, pp. 2418 to 2428, Nov. 2015, doi: 10.1109/JSAC.2015.2430281.
- [5] N. Pakanon, M. Chamchoy, and P. Supanakoon, "Study on Accuracy of Trilateration Method for Indoor Positioning with BLE Beacons," in *Proc. 2020 6th International Conference on Engineering, Applied Sciences and Technology (ICEAST)*, Chiang Mai, Thailand, 2020, pp. 1 to 4, doi: 10.1109/ICEAST50382.2020.9165464.
- [6] A. De Blas and D. Lopez de Ipina, "Improving Trilateration for Indoors Localization Using BLE Beacons," in *Proc. 2017 2nd International Multidisciplinary Conference on Computer and Energy Science (SpliTech)*, Split, Croatia, 2017, pp. 1 to 6.
- [7] R. Harle, "A Survey of Indoor Inertial Positioning Systems for Pedestrians," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 3, pp. 1281 to 1293, 2013, doi: 10.1109/SURV.2012.121912.00075.
- [8] L. Liu, B. Li, L. Yang, and T. Liu, "Real Time Indoor Positioning Approach Using iBeacons and Smartphone Sensors," *Applied Sciences*, vol. 10, no. 6, Art. no. 2003, 2020, doi: 10.3390/app10062003.
- [9] J. Robesaat, P. Zhang, M. Abdelaal, and O. Theel, "An Improved BLE Indoor Localization with Kalman Based Fusion: An Experimental Study," *Sensors*, vol. 17, no. 5, Art. no. 951, 2017, doi: 10.3390/s17050951.
- [10] F. Jamil, N. Iqbal, S. Ahmad, and D. H. Kim, "Toward Accurate Position Estimation Using Learning to Prediction Algorithm in Indoor Navigation," *Sensors*, vol. 20, no. 16, Art. no. 4410, 2020, doi: 10.3390/s20164410.
- [11] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35 to 45, 1960, doi: 10.1115/1.3662552.
- [12] Android Developers, "Bluetooth permissions," Android Developers. Accessed: May 31, 2026. [Online]. Available: <https://developer.android.com/develop/connectivity/bluetooth/bt-permissions>
- [13] Android Developers, "Motion sensors," Android Developers. Accessed: May 31, 2026. [Online]. Available: https://developer.android.com/develop/sensors-and-location/sensors/sensors_motion

[14] Android Developers, “Sensors overview,” Android Developers. Accessed: May 31, 2026. [Online]. Available: https://developer.android.com/develop/sensors-and-location/sensors/sensors_overview

[15] Kontakt.io, “Configure Proximity Manager,” Kontakt.io Developer Center. Accessed: May 31, 2026. [Online]. Available: <https://developer.kontakt.io/docs/dev-ctr-sdks/ZG9jOjQ3NDU0ODU0-configure-proximity-manager>

[16] Kontakt.io, “ProximityManager,” Kontakt.io Android SDK 4.0.2 Javadoc. Accessed: May 31, 2026. [Online]. Available: <https://kontaktio.github.io/kontakt-android-sdk/4.0.2/Javadoc/com/kontakt/sdk/android/ble/manager/ProximityManager.html>

Appendix A. Evaluation Metrics and Supporting Tables

Table A1. Additional sample level metrics for evaluation-eligible static samples.

Metric	Fused / rendered position	Raw accepted BLE baseline
Median error	0.6188 m	0.6777 m
Standard deviation	0.4578 m	0.4778 m
Evaluation eligible samples	46	46
Valid static evaluation sessions	9	9
Logged sessions / samples in full dataset	28 sessions / 241 samples	Context only

Appendix B. Configuration Summary

The following JSON derived values summarize the final frozen calibration used for the controlled evidence set. The full active configuration is included in Appendix C.

Table B1. Main values from current_test.json.

JSON path	Value
room.width	2.50
room.height	2.50
model.pathLossExponent	2.0
model.headingOffsetDegrees	146.0
model.stepLengthMeters	0.62
model.minStepLengthMeters	0.35
model.maxStepLengthMeters	0.72
model.rssiAverageWindowSize	3
model.bleMinDistanceMeters	0.08
model.bleDistanceMarginMeters	1.25
model.bleBoundaryClampMarginMeters	0.20
model.bleMaxJumpMeters	2.5
model.bleResidualRejectMeters	0.85

Appendix B.1 Heading Offset Calibration Note

The following note was moved from the main implementation chapter to keep the main body concise while preserving the calibration reasoning.

Heading offset calibration. The heading offset aligns the Android phone heading frame with the room/map coordinate frame used by the dashboard and positioning code. The app computes PDR displacement with $dx = \text{stepLength} * \sin(\text{correctedHeading})$ and $dy = \text{stepLength} * \cos(\text{correctedHeading})$, so the corrected heading must describe direction in the same x/y convention as the room map. A wrong heading offset can make PDR drift in the wrong direction even when step detection and step length are reasonable.

The most reliable way to calculate this offset is to use a known straight-line walking segment inside the same room. Two marked points are chosen with known meter coordinates $A = (x_1, y_1)$ and $B = (x_2, y_2)$. The user starts at A, walks straight toward B while holding the phone in the same way used during testing, and records the raw Android heading during the middle part of the walk. The room direction is then computed from the known displacement between the two points.

Because the project PDR equations use $dx = \text{stepLength} * \sin(\text{heading})$ and $dy = \text{stepLength} * \cos(\text{heading})$, the room direction is calculated as $\text{roomDirection} = \text{atan2}(x_2 - x_1, y_2 - y_1)$, not as the conventional mathematical $\text{atan2}(\text{deltaY}, \text{deltaX})$. The offset is then computed as $\text{headingOffset} = \text{roomDirection} - \text{averageRawHeading}$, with angular values normalized to either 0 to 360 degrees or -180 to 180 degrees. The raw headings should be averaged using circular averaging, because headings wrap around at 0/360 degrees.

For example, a calibration walk from $A = (0.00, 0.00)$ to $B = (2.50, 0.00)$ has $\text{deltaX} = 2.50$ and $\text{deltaY} = 0.00$, therefore $\text{roomDirection} = \text{atan2}(2.50, 0.00) = 90$ degrees. If the average raw Android heading while walking this direction is 304 degrees, then $\text{headingOffset} = 90 - 304 = -214$ degrees, which is equivalent to 146 degrees after normalization to the 0 to 360 degree range. This illustrates how the current 146.0 degree value can be justified as a room frame alignment parameter.

A stronger calibration procedure would repeat the same calculation over several straight walking segments in different directions and circular average the resulting offsets. This would reduce sensitivity to hand angle, magnetic disturbances, small walking angle errors, and sensor noise. In

the current thesis implementation, however, the 146.0 degree value is reported as empirically tuned during development rather than formally optimized through a dedicated calibration experiment.

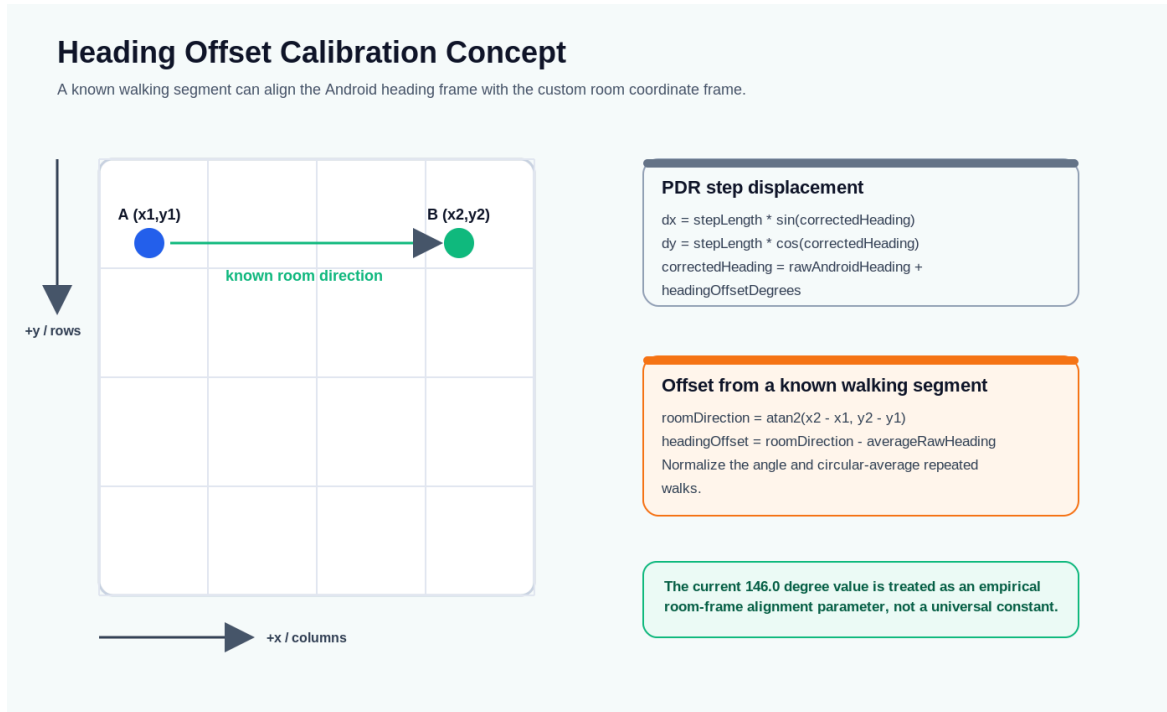


Figure B1. Heading offset calibration concept. The offset rotates Android sensor heading into the room-coordinate frame used by PDR and fusion.

Appendix C. Full current_test.json

This appendix contains the active calibration file in a readable format. The values are also summarized in Appendix B, but the listing below preserves the actual JSON structure used by the Android application at app/src/main/assets/CalibrationTests/current_test.json.

Table C1. Structured view of room and model parameters from current_test.json.

JSON path	Value	Meaning
room.width	2.5	Active room width in meters.
room.height	2.5	Active room height in meters.
model.pathLossExponent	2.0	Exponent n used by the log distance RSSI to distance model.
model.headingOffsetDegrees	146.0	Empirical phone to room heading frame alignment value.
model.stepLengthMeters	0.62	Nominal PDR step length.
model.minStepLengthMeters	0.35	Minimum adaptive PDR step length.
model.maxStepLengthMeters	0.72	Maximum adaptive PDR step length.
model.minStepIntervalMs	650	Minimum interval between accepted step events.
model.customStepPeakThreshold	1.4	Acceleration peak threshold for custom step detection fallback.
model.customStepResetThreshold	0.35	Reset threshold used by the custom step detector.
model.customStepMaxGyro	0.95	Maximum gyroscope magnitude for custom step acceptance.

model.customStepMaxAbsPitchDegrees	35.0	Maximum absolute phone pitch for custom step acceptance.
model.customStepMaxAbsRollDegrees	35.0	Maximum absolute phone roll for custom step acceptance.
model.rssiAverageWindowSize	3	Number of latest RSSI samples averaged for BLE distance estimation.
model.bleMinDistanceMeters	0.08	Minimum accepted BLE distance estimate.
model.bleDistanceMarginMeters	1.25	Extra margin added to the room diagonal for BLE distance gating.
model.bleBoundaryClampMarginMeters	0.2	Maximum tolerated outside room solve before rejection.
model.bleMaxJumpMeters	2.5	Maximum accepted jump from the previous BLE fix.
model.bleResidualRejectMeters	0.85	Residual threshold used to reject inconsistent BLE solves.

Table C2. Floor plan matrix. A value of 1 marks a configured transmitter cell and 0 marks an empty cell.

row/col	col 0	col 1	col 2	col 3	col 4
row 0	1	0	0	0	0
row 1	0	0	0	0	0
row 2	0	0	1	0	0
row 3	0	0	0	0	0
row 4	1	0	0	0	1

Table C3. Beacon definitions from current_test.json.

ID	UUID	Major	Minor	Grid row	Grid col	RSSI at 1 m
go6G	f7826da6-4fa2-4e98-8024-bc5b71e0893e	20642	50360	0	0	-80
yiBb	f7826da6-4fa2-4e98-8024-bc5b71e0893e	43294	40290	4	0	-86
SamA15	f7826da6-4fa2-4e98-8024-bc5b71e0893e	20641	50361	4	4	-80
Red10	f7826da6-4fa2-4e98-8024-bc5b71e0893e	20643	50363	2	2	-80

Listing C1. Complete current_test.json in compact readable form.

1	{
2	"room": { "width": 2.5, "height": 2.5 },
3	"model": {
4	"pathLossExponent": 2,
5	"headingOffsetDegrees": 146,
6	"stepLengthMeters": 0.62,
7	"minStepLengthMeters": 0.35,
8	"maxStepLengthMeters": 0.72,
9	"minStepIntervalMs": 650,
10	"customStepPeakThreshold": 1.4,
11	"customStepResetThreshold": 0.35,
12	"customStepMaxGyro": 0.95,
13	"customStepMaxAbsPitchDegrees": 35,
14	"customStepMaxAbsRollDegrees": 35,
15	"rssiAverageWindowSize": 3,
16	"bleMinDistanceMeters": 0.08,
17	"bleDistanceMarginMeters": 1.25,
18	"bleBoundaryClampMarginMeters": 0.2,
19	"bleMaxJumpMeters": 2.5,
20	"bleResidualRejectMeters": 0.85
21	},
22	"floorPlan": [
23	[1, 0, 0, 0, 0],

```

24     [0, 0, 0, 0, 0],
25     [0, 0, 1, 0, 0],
26     [0, 0, 0, 0, 0],
27     [1, 0, 0, 0, 1]
28 ],
29 "beacons": [
30     { "id": "go6G", "uuid": "f7826da6-4fa2-4e98-8024-bc5b71e0893e", "major":
20642, "minor": 50360, "position": { "row": 0, "col": 0 }, "rssi1Meter": -80 },
31     { "id": "yiBb", "uuid": "f7826da6-4fa2-4e98-8024-bc5b71e0893e", "major":
43294, "minor": 40290, "position": { "row": 4, "col": 0 }, "rssi1Meter": -86 },
32     { "id": "SamA15", "uuid": "f7826da6-4fa2-4e98-8024-bc5b71e0893e", "major":
20641, "minor": 50361, "position": { "row": 4, "col": 4 }, "rssi1Meter": -80 },
33     { "id": "Red10", "uuid": "f7826da6-4fa2-4e98-8024-bc5b71e0893e", "major":
20643, "minor": 50363, "position": { "row": 2, "col": 2 }, "rssi1Meter": -80 }
34 ]
35 }

```

Appendix D. Source Code References and Selected Snippets

The thesis does not include full Java source files in the main report. Instead, the implementation is documented through selected small snippets, tables of file responsibilities, and references to the source files. The full implementation is submitted through the project repository referenced in Appendix E.

RSSI distance conversion (bleBeacon.java)

```
return (float) Math.pow(10d, (txPower - rssi) / (10.0 * n));
```

Weighted multilateration weight floor (TrilaterationUtils.java)

```
double effectiveDist = Math.max(ri, MIN_WEIGHT_DIST_M);

double wi = 1.0 / (effectiveDist * effectiveDist);
```

Heading frame alignment (SensorManager.java)

```
double expectedHeadingRad = Math.atan2(deltaXMeters, deltaYMeters);

headingOffsetRad = normalizeAngle(expectedHeadingRad - rawHeadingRad);
```

Manual anchor evaluation policy (MainActivity.java)

```
// Map taps are tracker anchors. They reset Kalman exactly, but their

// ground-truth is not evaluation evidence because the state is forced to it.
```

Appendix D.1 Runtime Processing Flow

The detailed runtime sequence moved out of the main body is listed below for reproducibility and implementation traceability.

1. Load calibration data from CalibrationTests/current_test.json.
2. Initialize the Kontakt.io SDK and Android BLE scanning after permissions and device prerequisites are satisfied.

3. Accept only BLE transmitters that match UUID, major, and minor values defined in the active calibration.
4. Maintain recent RSSI history for each live transmitter and convert averaged RSSI values into distance estimates.
5. Filter unsuitable distance estimates and solve a BLE position using weighted multilateration when at least three valid transmitters are available.
6. Reject unreliable BLE positions using residual, boundary, and jump checks.
7. Pass accepted BLE fixes into the sensor-fusion component.
8. Use phone sensors and PDR to predict relative movement between BLE fixes.
9. Fuse PDR predictions and accepted BLE corrections using the Kalman filter.
10. Render the selected tracking mode on the dashboard and write experiment logs.

Appendix D.2 Detailed Implementation and Log Tables

The following tables were moved from the main body so that Chapter 5 focuses on system design and implementation flow, while detailed file responsibilities, gates, input roles, and log outputs remain available for verification.

Table D1. Main implementation files and responsibilities.

Component	Main responsibility
MainActivity.java	Android lifecycle, permissions, BLE scanning, raw BLE debug scan, positioning mode selection, manual anchors, dashboard updates, recording, and log export.
InitApp.java	Kontakt.io SDK initialization, ProximityManager creation, scan configuration, and background/UI handler setup.
JsonOps.java	Parsing and validating the active calibration JSON, including room dimensions, floor plan, beacon definitions, and model parameters.
BeaconManager.java / bleBeacon.java	Maintaining the live beacon set, accepting only configured transmitters, storing recent RSSI history, and computing RSSI based distance estimates.
PositioningEngine.java	BLE positioning decisions: missing beacon reporting, distance gates, residual checks, room-boundary checks, and jump rejection.
TrilaterationUtils.java	Weighted least squares BLE multilateration using accepted beacon distances and known coordinates.
SensorManager.java	Android sensor registration, heading estimation, step detection, PDR movement, motion state detection, and Kalman fusion policy.
KalmanPositionFilter.java	Framework-free 2D constant-velocity Kalman filter for fused position estimation.

Component	Main responsibility
TrackingDashboardView.java	Live dashboard UI, room map, mode controls, adaptive toggle, recording controls, labels, and export action.
UserPositionHistory.java / UserPositionSample.java	Timestamped CSV/JSON session logging, sample schema, ground truth fields, evaluation eligibility, and summary context.

Table D2. BLE acceptance gates and rejection checks.

Gate/check	Current value or behavior
Minimum BLE distance	0.08 m
Maximum solve distance	room diagonal + 1.25 m distance margin
Minimum valid beacons	3 transmitters for 2D positioning
Residual rejection threshold	0.85 m
Boundary clamp margin	0.2 m
Maximum jump	2.5 m
Fewer than 3 valid transmitters	No BLE fix; fallback to existing fused/PDR state when possible

Table D3. Fusion inputs and their role in the final rendered position.

Input to fusion	How it is used
Manual map anchor	Resets the Kalman filter to a known position and resets BLE jump history. Not evaluation evidence.
Accepted BLE fix	Corrects the predicted Kalman position using measurement noise based on motion state, beacon count, and optional quality score.
PDR step	Predicts relative displacement using step length and corrected heading.
Motion state	Adjusts process behavior and measurement confidence between stationary and moving states.
Room dimensions	Clamps fused output to the valid room-boundary.

Table D4. Main categories stored in runtime CSV/JSON logs.

Logged field category	Examples
Session metadata	sessionId, schema version, start timestamp, recording state, trial label
Rendered position	renderXMeters, renderYMeters, renderRow, renderCol
BLE position	bleXMeters, bleYMeters, BLE row/col, hasBleFix
Ground truth	groundTruthXMeters, groundTruthYMeters, truthSource, evaluationEligible
Errors	errorMeters, bleErrorMeters when ground truth exists
Diagnostics	liveCount, filteredCount, BLE quality, PDR quality, RSSI noise, residual RMS/max, motion state

Table D5. Thesis report-generation outputs.

Generated output	Purpose
thesis_sessions_summary.csv	One row per session with sample counts, BLE fix ratio, evaluation sample counts, and accuracy metrics when valid ground truth exists.
thesis_samples.csv	One row per position sample with render/BLE/truth coordinates, eligibility flags, errors, diagnostics, and labels.
<sessionId>_path.svg	Path visualization of rendered samples, BLE fixes, anchors, and beacon positions.
<sessionId>_error.svg	Error over time plot generated only when evaluation eligible ground truth samples exist.
evaluation_annotations.csv	Optional manual annotation input keyed by sessionId and sampleIndex for final ground truth labeling.

Appendix E. Repository and Submission Notes

Table E1. Hardware/device setup used during development and final experiments.

Device / component	Role in the project	Notes
Ulefone Armor 22	Receiver / scanner phone	Android 13; used as the device running OIPASFT during final experiments.
Kontakt.io physical beacons	Physical BLE/iBeacon transmitters	Used as physical transmitters during development and testing where available. The specific beacon model is listed in Appendix E.
Samsung Galaxy A15	Simulated BLE advertiser	Android 14; used as a BLE simulator advertiser where applicable.
Xiaomi Redmi Note 10	Simulated BLE advertiser	Android 13; used as a BLE simulator advertiser where applicable.

- Project repository: <https://github.com/MikeSamaritakis/Thesis-Submission-Michail-Samaritakis>
- Full Java source files are not reproduced in the report body. The report documents the implementation through architecture descriptions, file responsibility tables, selected snippets, and source file references.
- The physical beacon model used during development/testing where available was the Kontakt.io Smart Beacon SB16-2. The final transmitter setup also used Android phones as simulated BLE advertisers where applicable.
- Manual anchors are initialization/reset events and are not independent accuracy samples.
- Walking routes are qualitative only because continuous independent ground-truth was not recorded.
- Adaptive tuning is implemented in the prototype but was not tested as final thesis evidence.